Restaurant Management System

Requirements Analysis Document

Version 5

CSCI 4711 Software Engineering
Spring 2023
Augusta University

Team Members

Randi Williams
Teke Roberts
Tom Rusch
Ward Tyre

Table of Contents

1. INTRODUCTION 4
1.1 SCOPE OF SYSTEM 4
2. REQUIREMENTS OF SYSTEM 5
2.1 FUNCTIONAL REQUIREMENTS 5
2.2 NON-FUNCTIONAL REQUIREMENTS 6
2.3 USE CASES 7
2.4 USE CASE DESCRIPTIONS 8
2.5 REQUIREMENTS ANALYSIS 14
3 USER INTERFACE MOCKUPS 21
3.1 LOGIN 21
3.2 LOGINFAILED 22
3.2 ADDITEM 23
3.3 ENTERORDER 25
3.4 LOGOUT 27
4 OBJECT DESIGN 28
4.1 OBJECT RELATIONSHIP 28
4.2 DETAILED CLASS DESIGN 31
4.3 DATABASE DESIGN 32
5 SYSTEM DESIGN 33
5.1 SUBSYSTEM DECOMPOSITION 33
6 APPENDIX 34
6.1 APPENDIX A – SOURCE CODE 34

1. INTRODUCTION

1.1 SCOPE OF SYSTEM

This system is designed for an owner of a restaurant to manage the day-to-day operation of their business. It includes actors such as *Manager*, which represents those higher-level employees, and *Waiter*, which represents the employees that serve customers.

The system will keep track of both actors by keeping track of account information, current menu items, and items that have been ordered. It also has access to a database of employee account information. The two types of employees will interact with the system through the UI, shown later in this document.

The system includes functionality for verifying both types of users, as well as separate functions for the manager and waiter actors, respectively. Users logged in as *Managers* will be able to add menu items. Users logged in as *Waiter* will be able to enter orders from the tables they are serving. All data stores are internal and part of the system. The system is stand-alone, external interaction is solely with the employees that are authorized to use the system.

2. REQUIREMENTS OF SYSTEM

2.1 FUNCTIONAL REQUIREMENTS

- Startup: This functionality performs the initial configuration of the system including its database and user interface.

- Login – This function will allow employees using the system to sign in as either a Manager or Waiter

- AddItem – This function lets users logged in as Mangers to add items to the Restaurant's menu. Users will select the category of the new item, add a short description, and the set the price.

- EnterOrder – This function allows users logged in as Waiters to enter the orders of the tables that they are serving. They will input the table number and the menu items that were ordered by the customers.

- Logout – This function is used by both types of users to logout of the current logged account. It will take the system back to the intial login page

2.2 NON-FUNCTIONAL REQUIREMENTS

- Platform
    - Target operating system is Microsoft Windows
    - System is not web-based and should not utilize a web browser
    - System should be implemented using the C# programming language
- Security
    - All user input is validated for appropriateness and prevention of SQL injection attacks
    - User login information should not be stored as plaintext in the database
- Usability
    - System should implement a graphical user interface

2.3 USE CASES

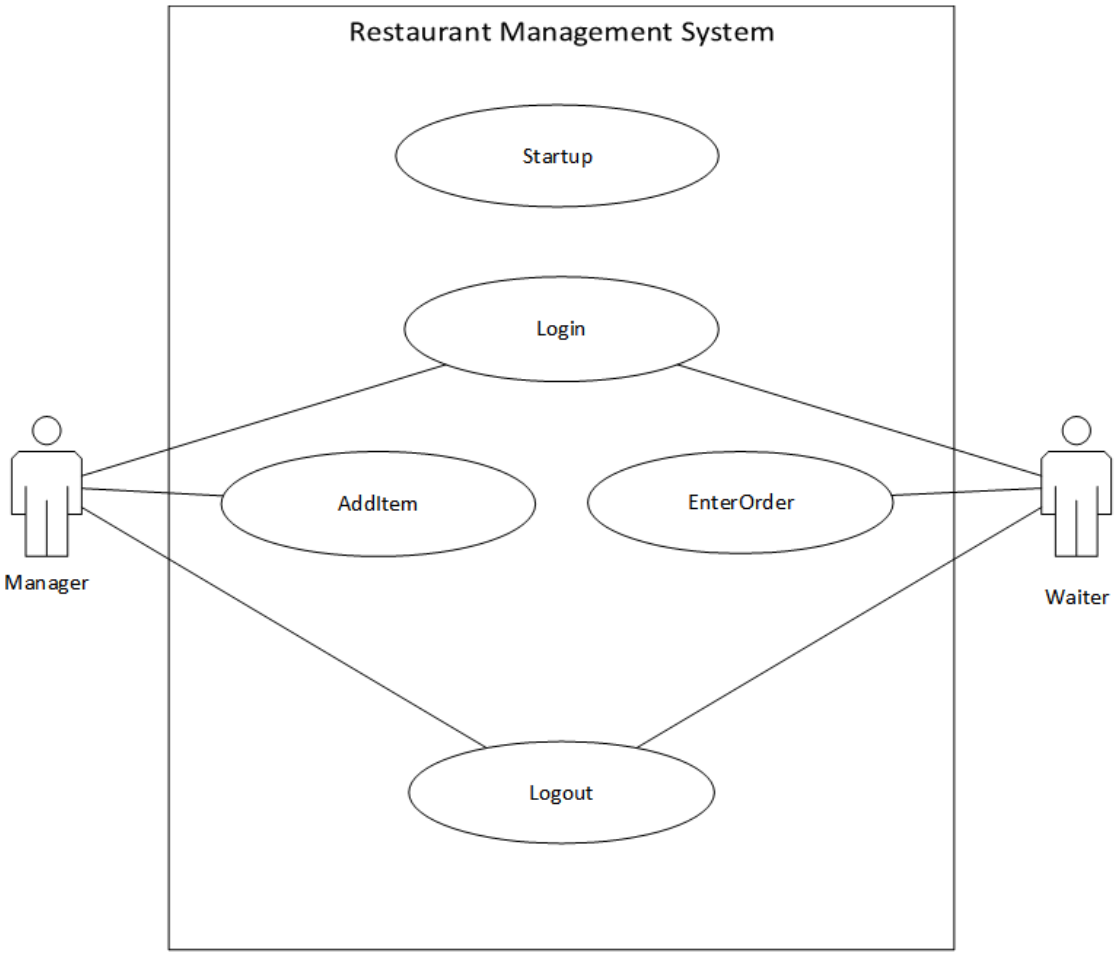


Figure 2.1: use case diagram

2.4 USE CASE DESCRIPTIONS

Use case name	Startup
Participating actors	
Flow of events	1. This use case is initiated when the application is launched. 2. The system reads the configuration file, initializes the database, and displays the login page.
Pre-condition	–
Post-condition	– The login page is displayed.
Security requirements	– User credential is not stored as plaintext in the database.

Figure 2.2: Startup

Use case name	Login
Participating actors	Initiated by `Manager` or `Waiter`
Flow of events	**1.** After startup sequence concludes, the actor (either `manager` or `waiter`) is brought to the login page, where they input their login information. **2.** System validates type of actor based on their credentials, and if login is successful, they are brought to their respective landing page. Login event is saved to database.
Pre-condition	– Startup sequence must complete (database must be loaded, initialized etc.)
Post-condition	– If a `manager` logs in and their credentials are successfully validated, they are brought to the manager portal. – If a `waiter` logs in and their credentials are successfully validated, they are brought to the order taker menu.
Security requirements	– Login credentials must not be stored/accessed in plaintext. – User input is validated for proper format and appropriateness to prevent vulnerabilities via SQL injection

Figure 2.3: Login

Use case name	Login
Participating actors	Initiated by `Manager` or `Waiter`
Flow of events	1. After startup sequence concludes, the actor (either `manager` or `waiter`) is brought to the login page, where they input their login information. 2. System validates the type of actor based on their credentials. The credentials are not found in database, and access is denied to the system and the message 'Your username or password is incorrect' is displayed.
Pre-condition	− Startup sequence must complete (database must be loaded, initialized etc.)
Post-condition	− Login page displayed with error message: "Your username or password is incorrect"
Security requirements	− Login credentials must not be stored/accessed in plaintext. − User input is validated for proper format and appropriateness to prevent vulnerabilities via SQL injection

Figure 2.4: LoginFailed

Use case name	AddItem
Participating actors	Available to `manager` users only
Flow of events	1. Manager selects the ‘add item’ button from the ManagerMenu 2. System displays the ItemScreen.screen which allows the user to create a new menu item by adding a name, description, category, and price. 3. Manager enters the required information in the ItemScreen and submits the form. 4. System saves the new menu item to the database and displays the ManagerMenu with an updated list of menu items.
Pre-condition	− This use case requires a user to be logged in as a `manager`, and adding an item requires filling out the ‘add item’ screen with the item details
Post-condition	− The new menu item is added to the menu on the manager landing screen.
Security requirements	− The user should not be able to enter invalid data (e. g. non-numeric values for price) so as to not compromise the integrity of the data

Figure 2.5 AddItem

Use case name	EnterOrder
Participating actors	Initiated and accessible by `waiter` users
Flow of events	1. Use case is initiated when the waiter selects the ‘take order’ button from the WaiterMenu 2. System displays the EnterOrder page which contains the various fields relating to a new order including table number, items ordered, and any comments/special requests. 3. Waiter enters the order information in the EnterOrder page and submits the forms. 4. System saves the new order to the database and displays the WaiterMenu with an updated list of open orders.
Pre-condition	– This use case requires a user to be logged in as a `waiter`, and adding an order requires filling out the ‘enter order’ screen with the order details
Post-condition	– New order is displayed on the waiter landing screen after submission
Security requirements	– Order input must be validated to preserve data integrity (field type checking, etc.)

Figure 2.6 EnterOrder

Use case name	Logout
Participating actors	Initiated by `manager` and `waiter`
Flow of events	1. This function occurs when the user, either a manager or a waiter, selects 'log out' from either of their respective landing pages. 2. System saves the logout event to the database and displays the login screen with a message below the username and password fields displaying 'you have successfully logged out'
Pre-condition	− To access this function, a user must be successfully logged in as either a `waiter` or a `manager` and be viewing either of their respective landing pages.
Post-condition	− The login screen is displayed with the logout confirmation message below the input fields
Security requirements	− Data must be securely offloaded from the client, and the user must no longer have access to enter orders/add menu items.

Figure 2.7 Logout

2.5 REQUIREMENTS ANALYSIS

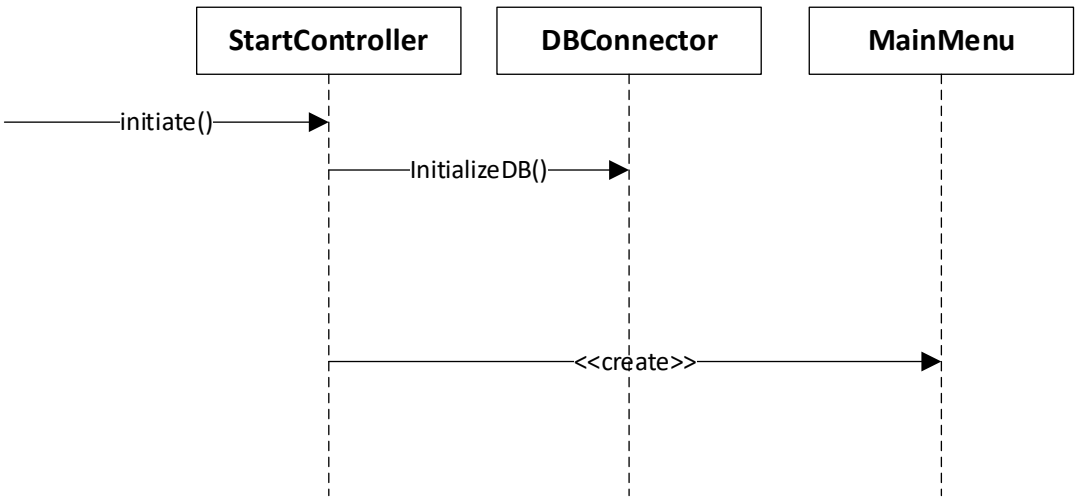


Figure 2.8 Startup

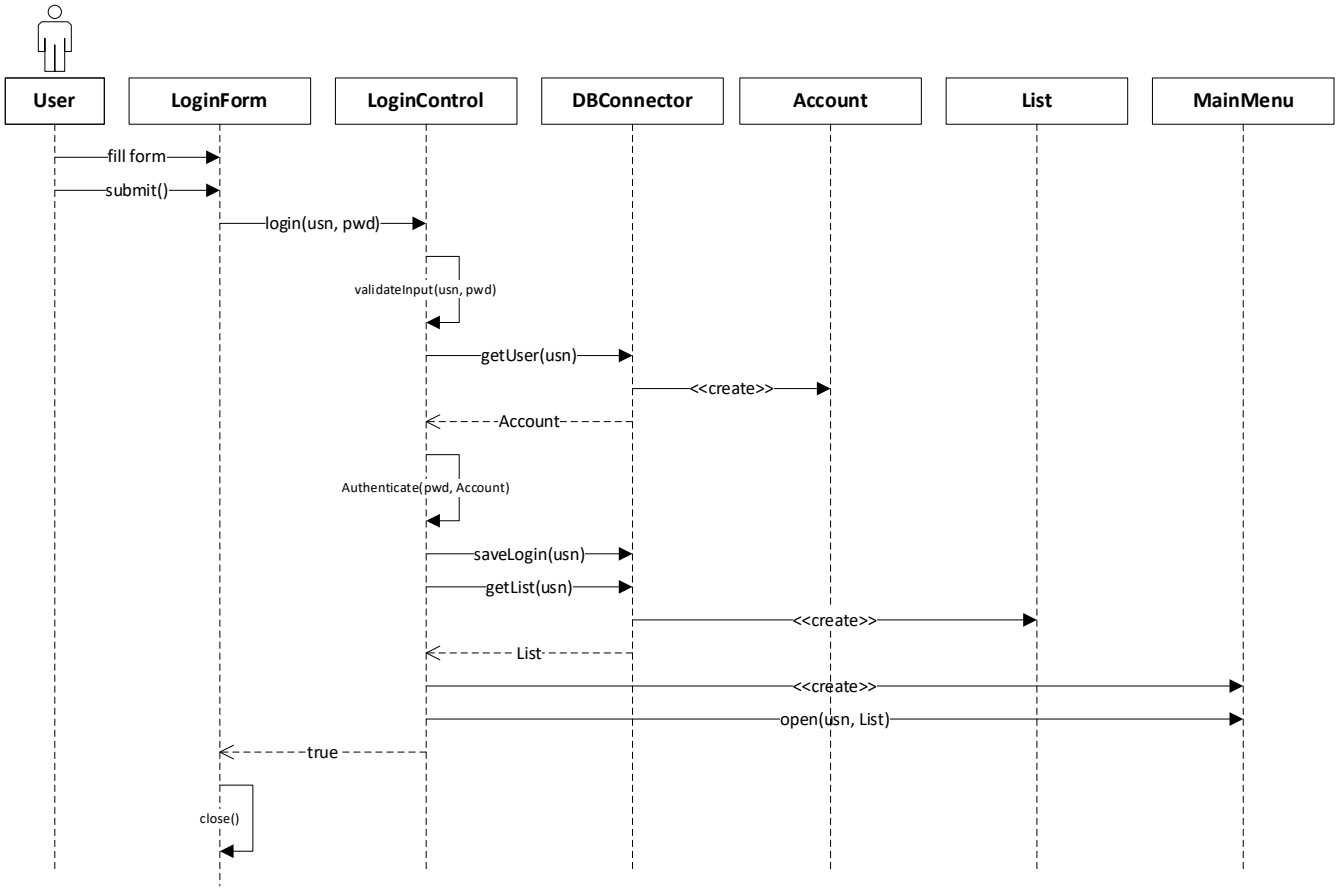


Figure 2.9 Log in Verified User

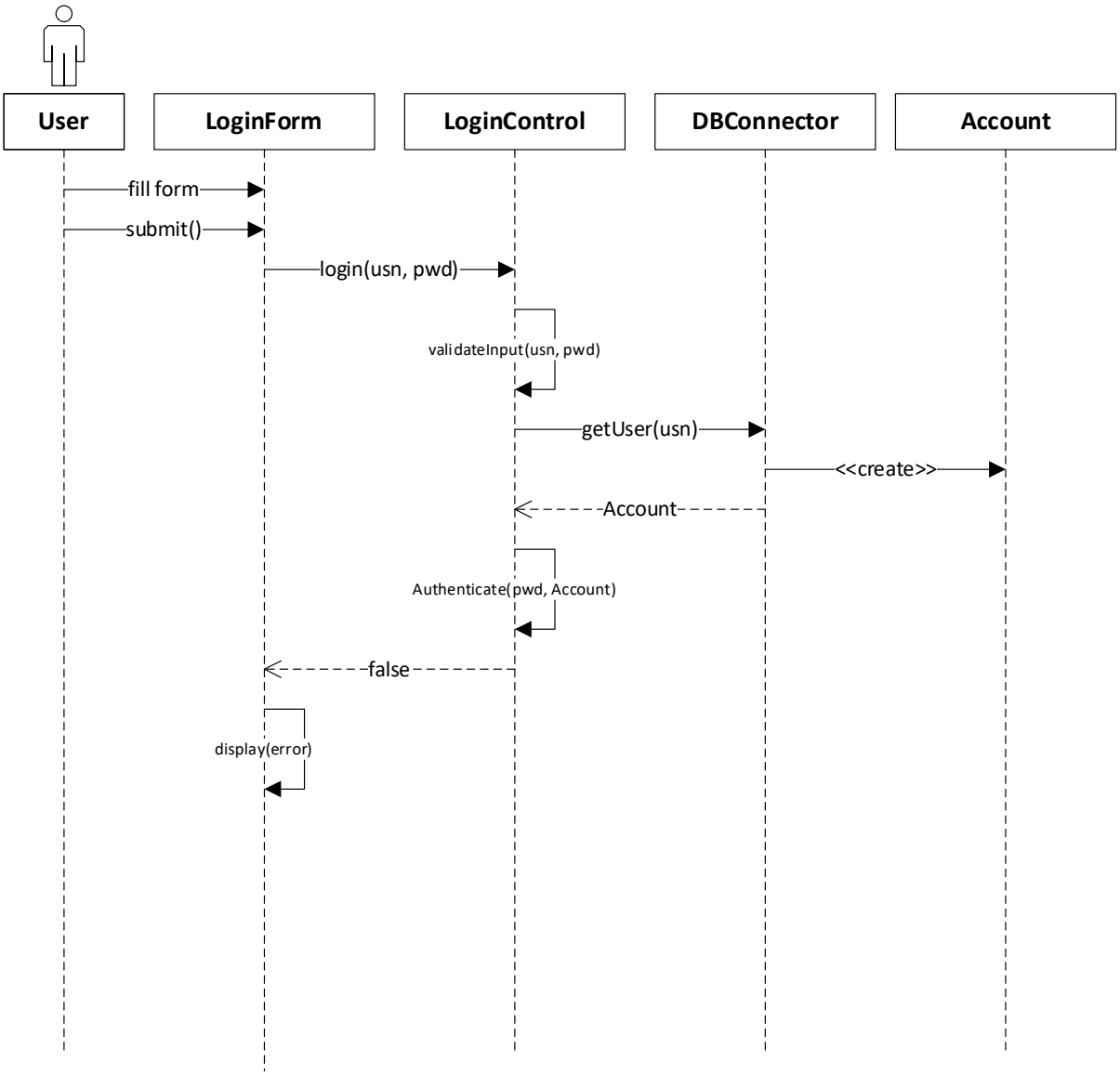


Figure 2.10 Login Failed

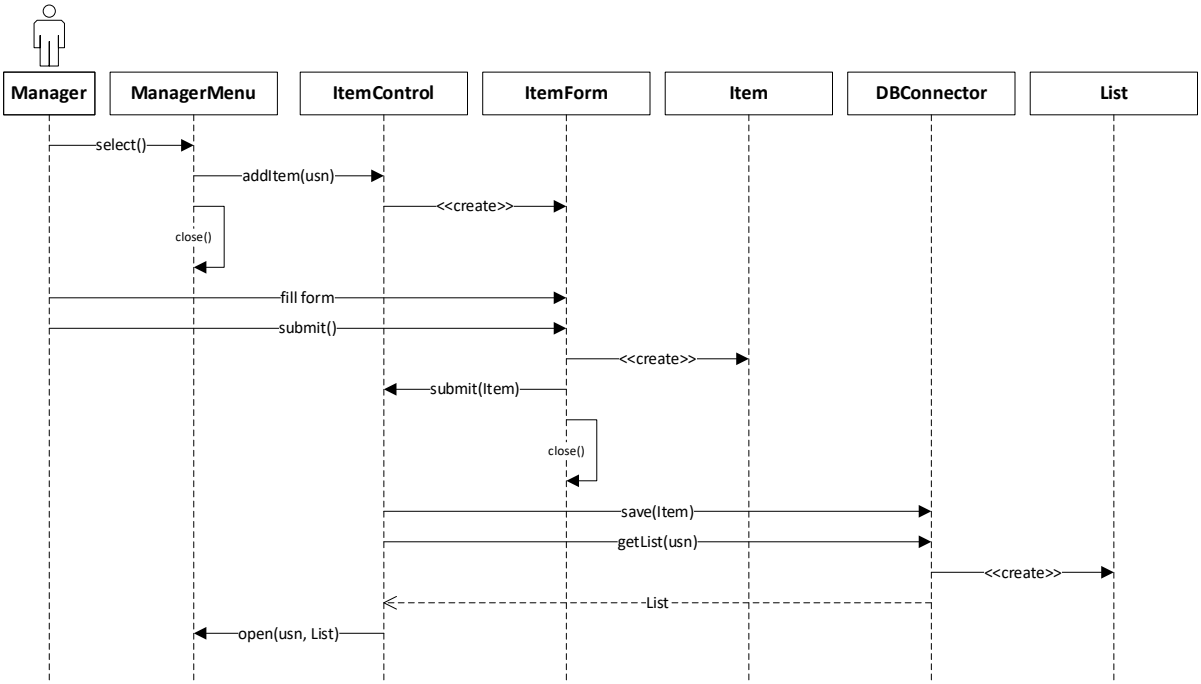


Figure 2.11 AddItem

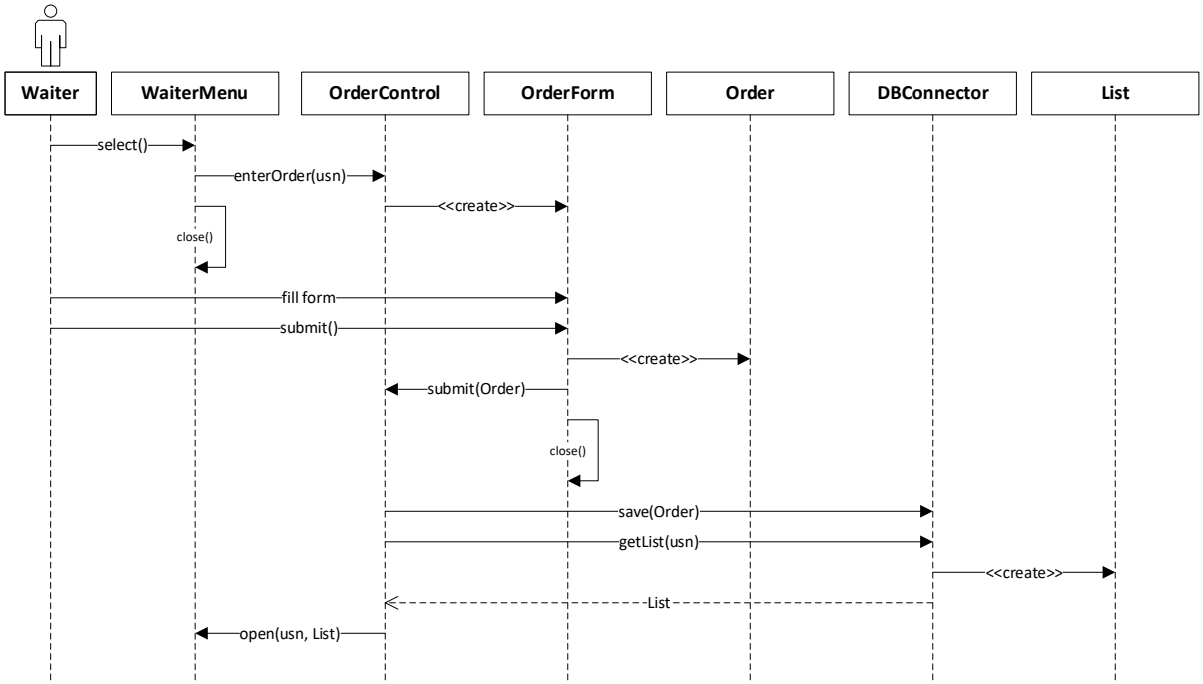


Figure 2.12 EnterOrder

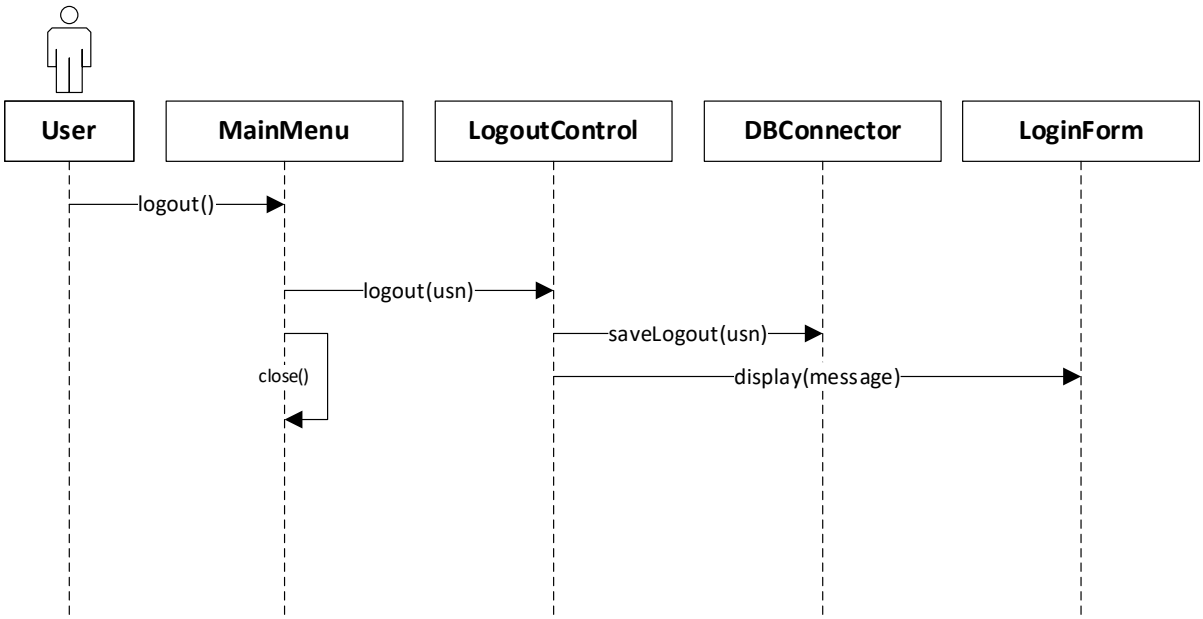


Figure 2.13 Logout

3 USER INTERFACE MOCKUPS

3.1 LOGIN

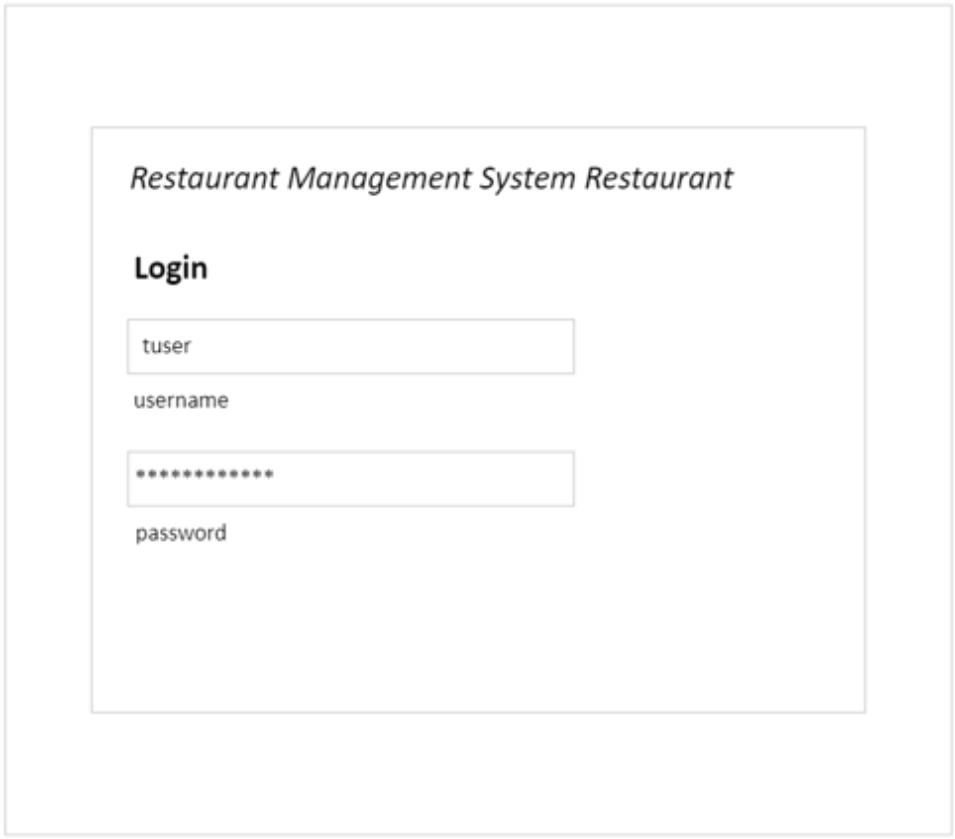


Figure 3.1: Login

3.2 LOGINFAILED

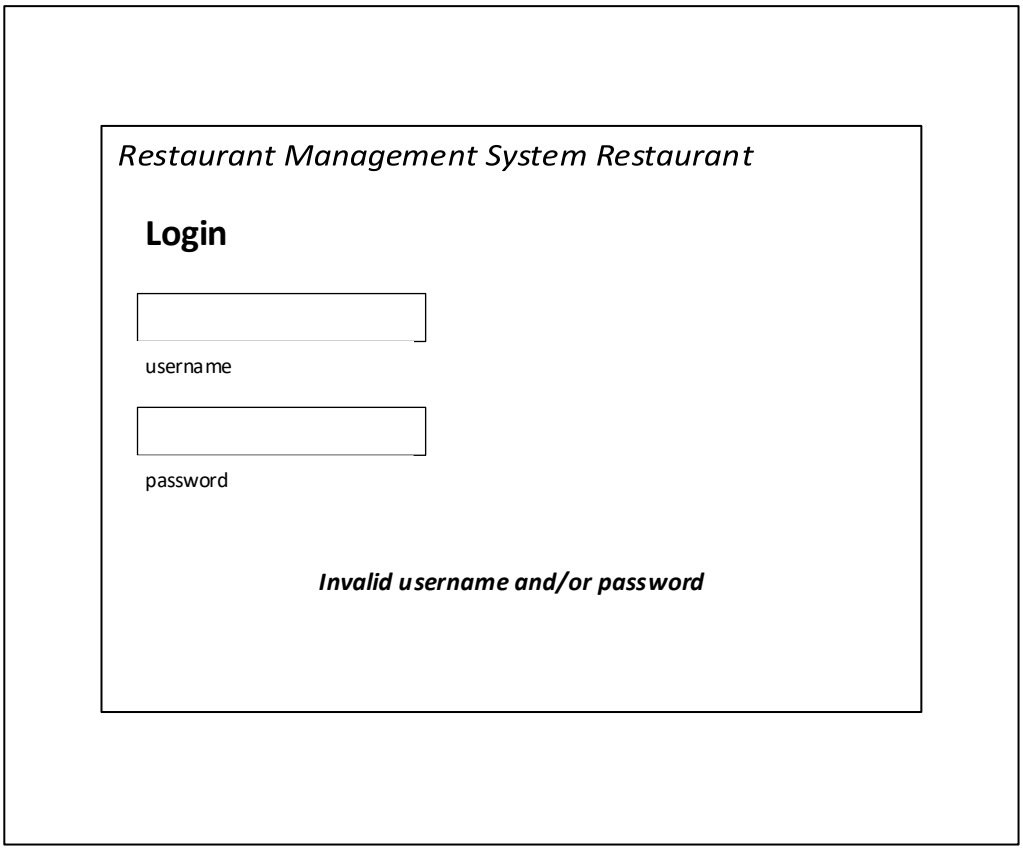


Figure 3.2: Login Failed

3.2 ADDITEM

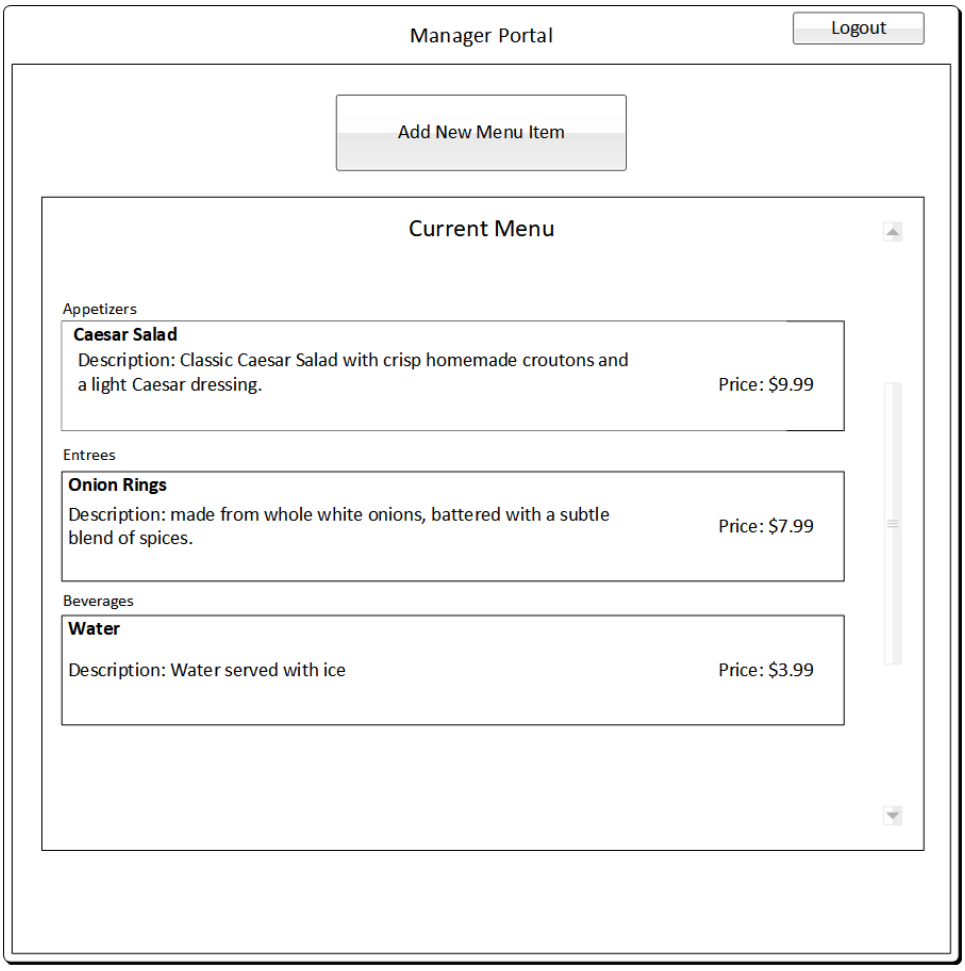


Figure 3.3 Manager Landing page



Figure 3.4: AddItem Menu

3.3 ENTERORDER

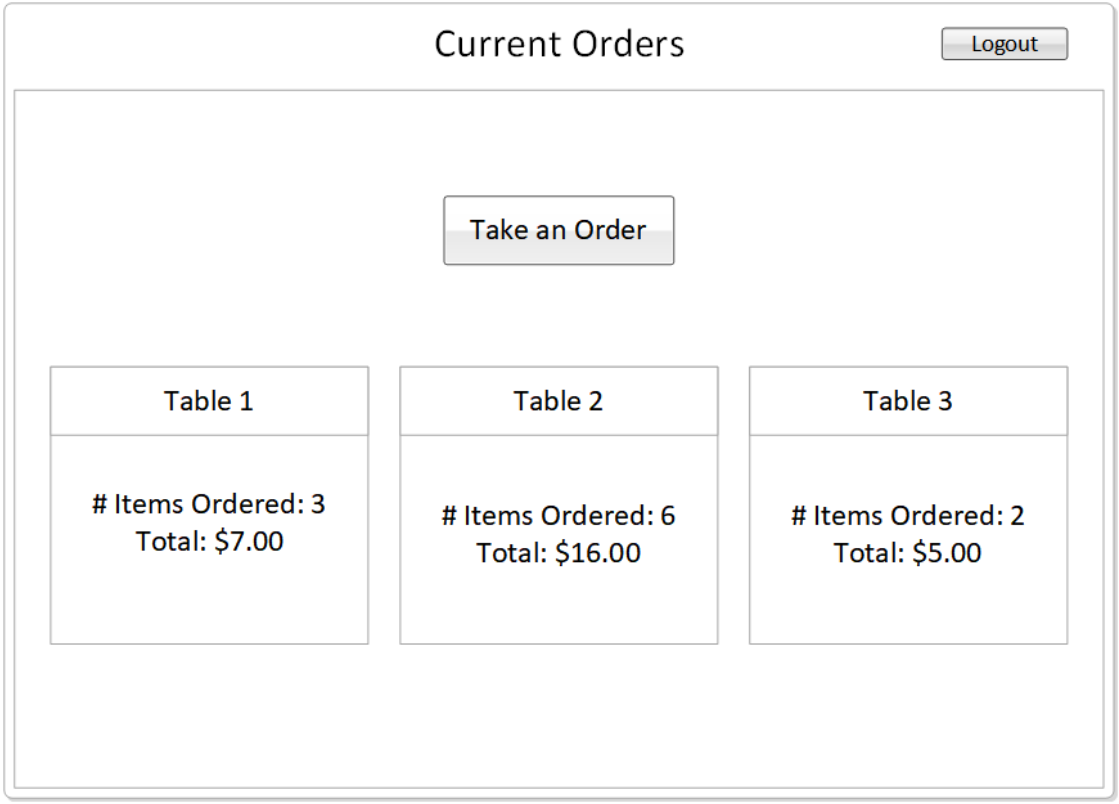


Figure 3.5: Waiter Landing page

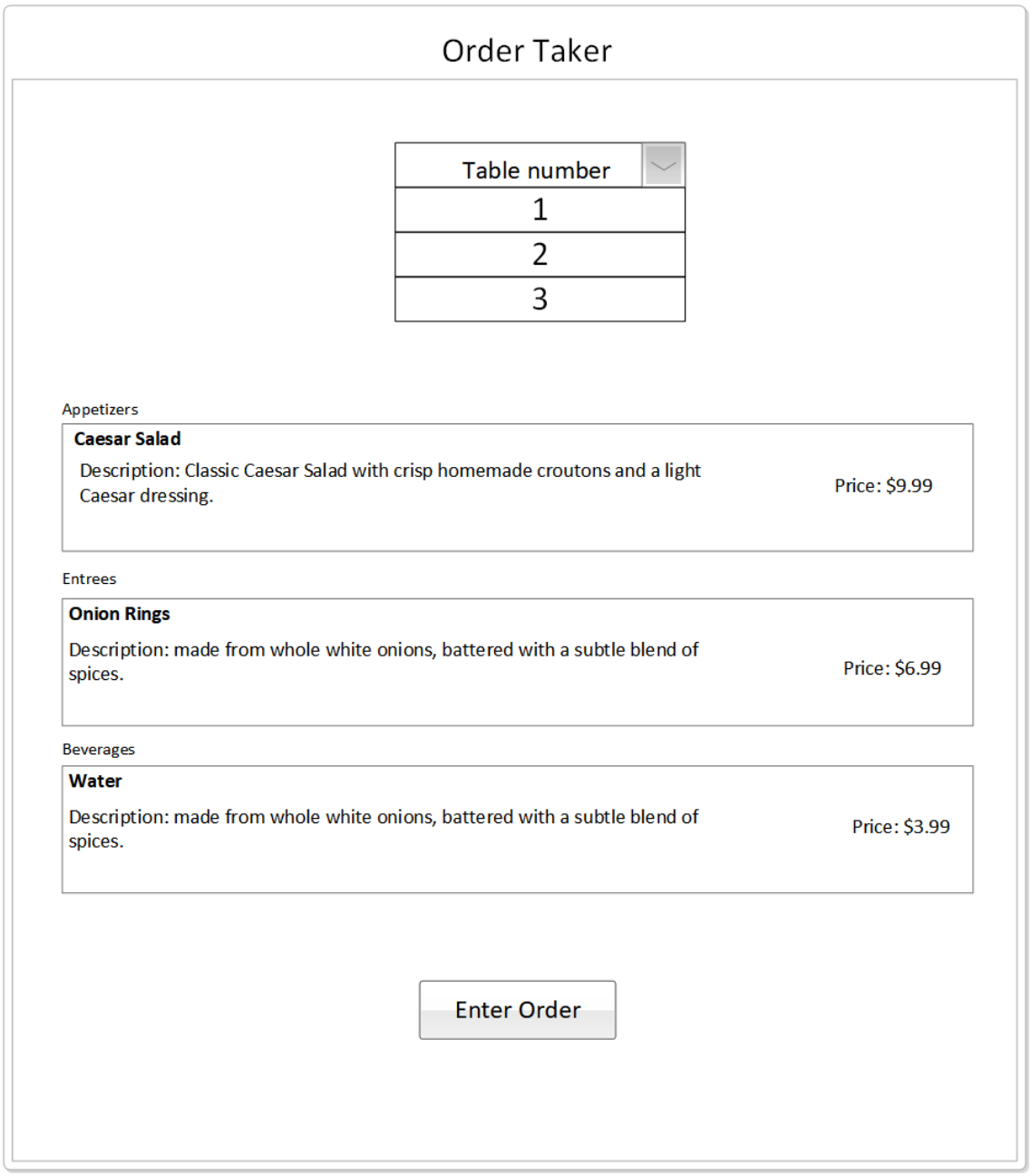


Figure 3.6 EnterOrder page

3.4 LOGOUT

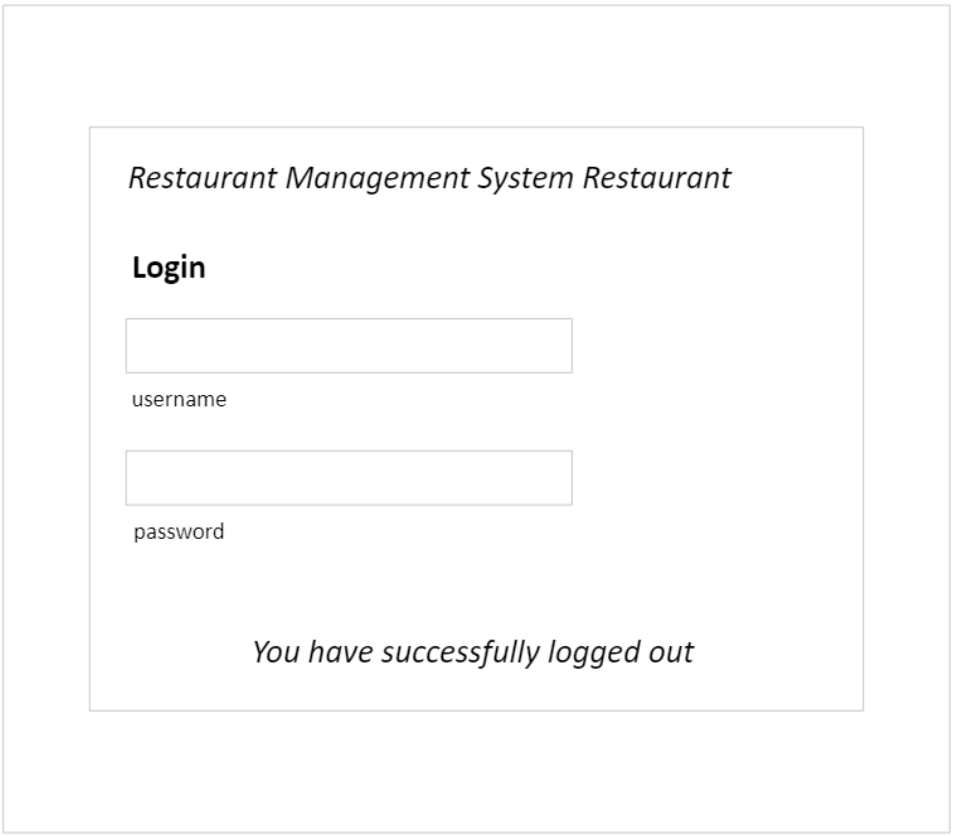


Figure 3.7 Logout Page

4 OBJECT DESIGN

4.1 OBJECT RELATIONSHIP

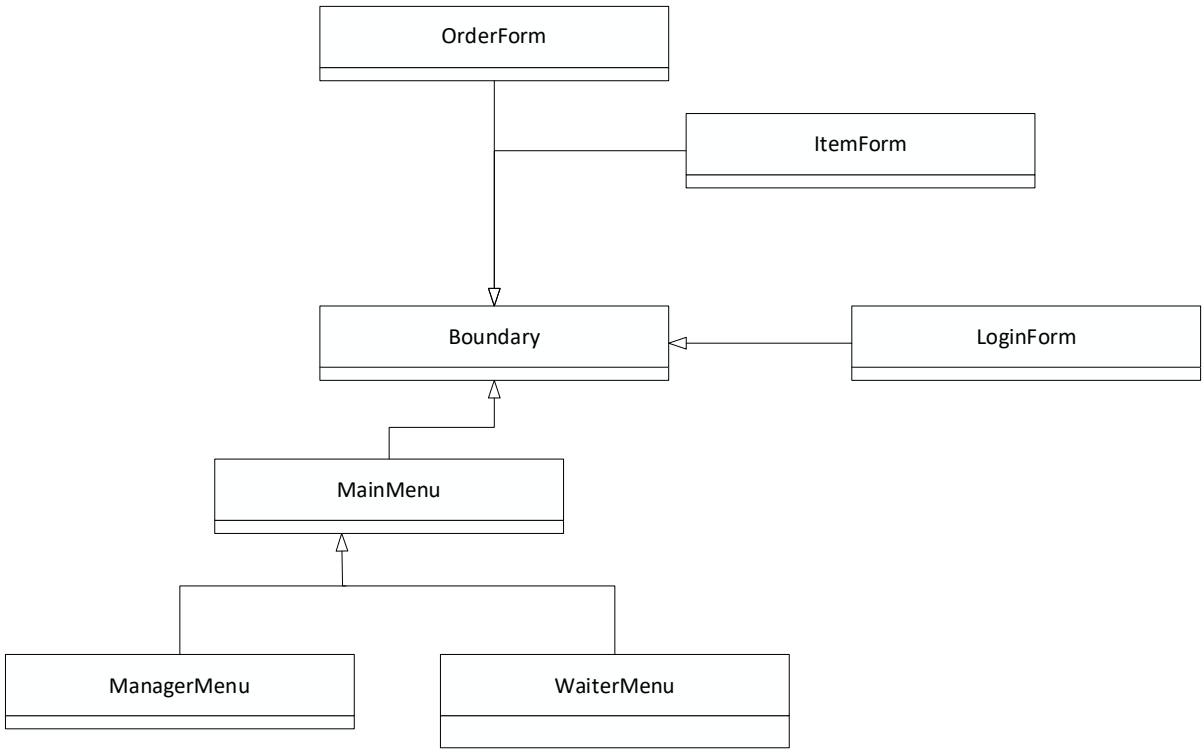


Figure 4.1 Class Diagram: Boundary

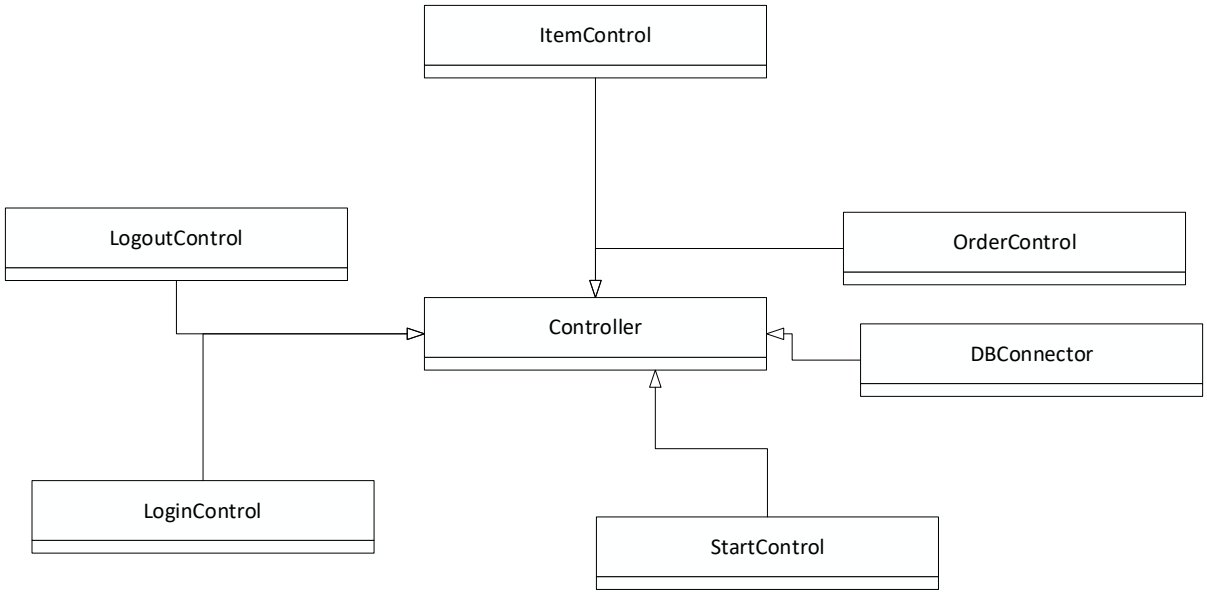


Figure 4.2 Class Diagram: Control

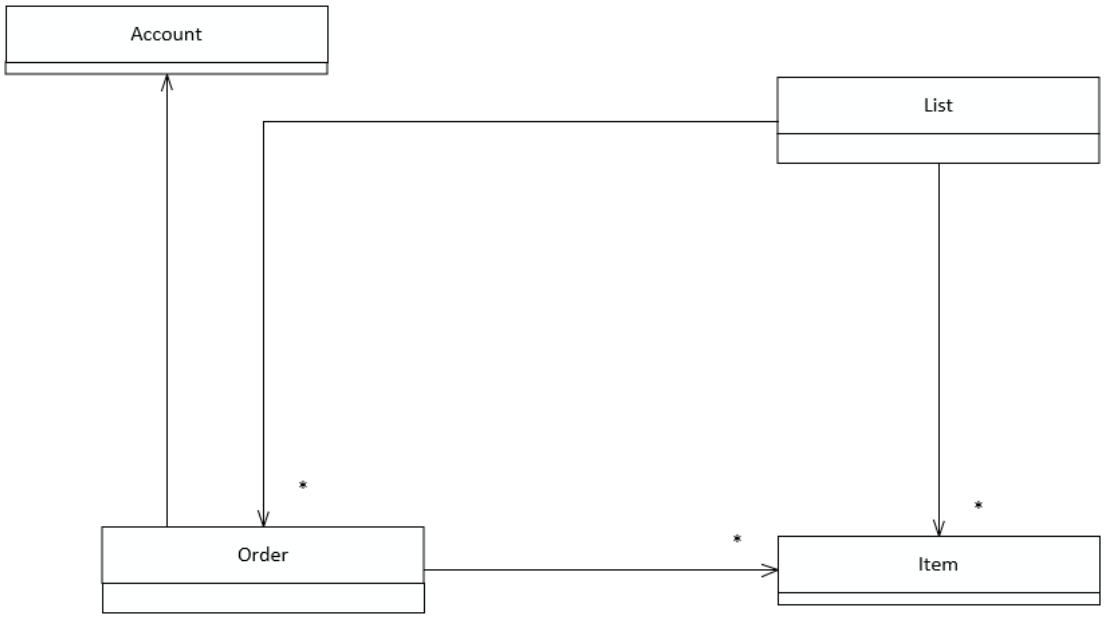


Figure 4.3 Class Diagram: Entity

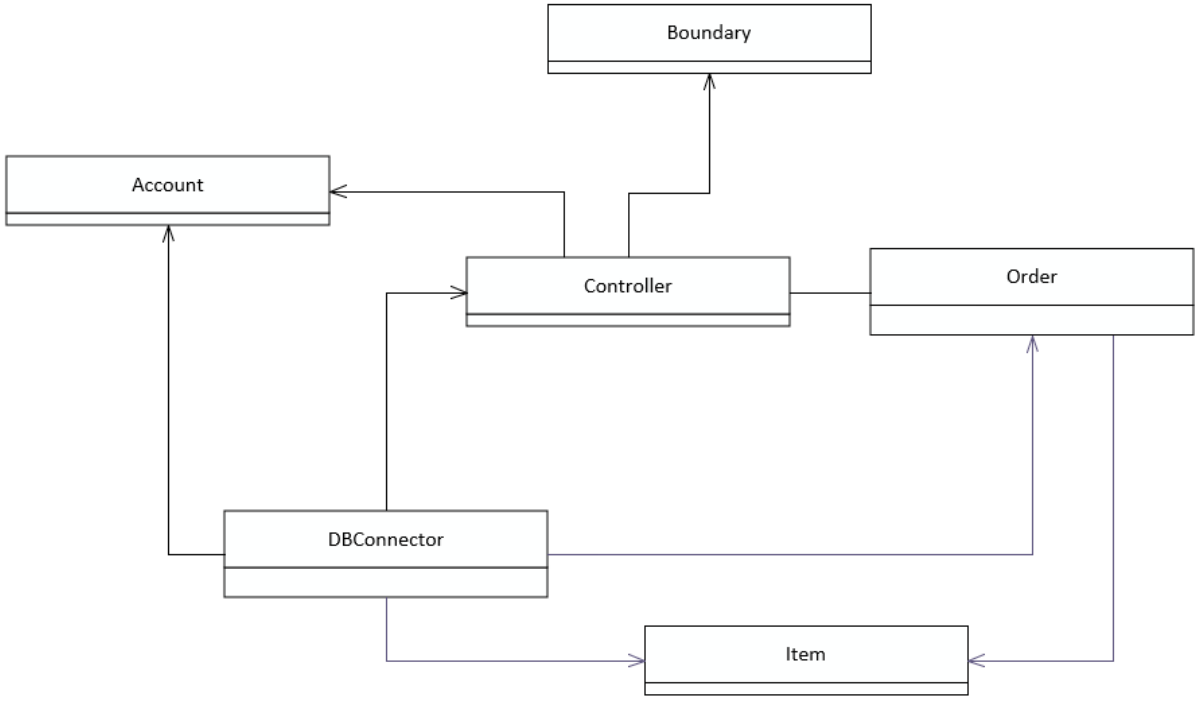


Figure 4.4 Class Diagram

4.2 DETAILED CLASS DESIGN

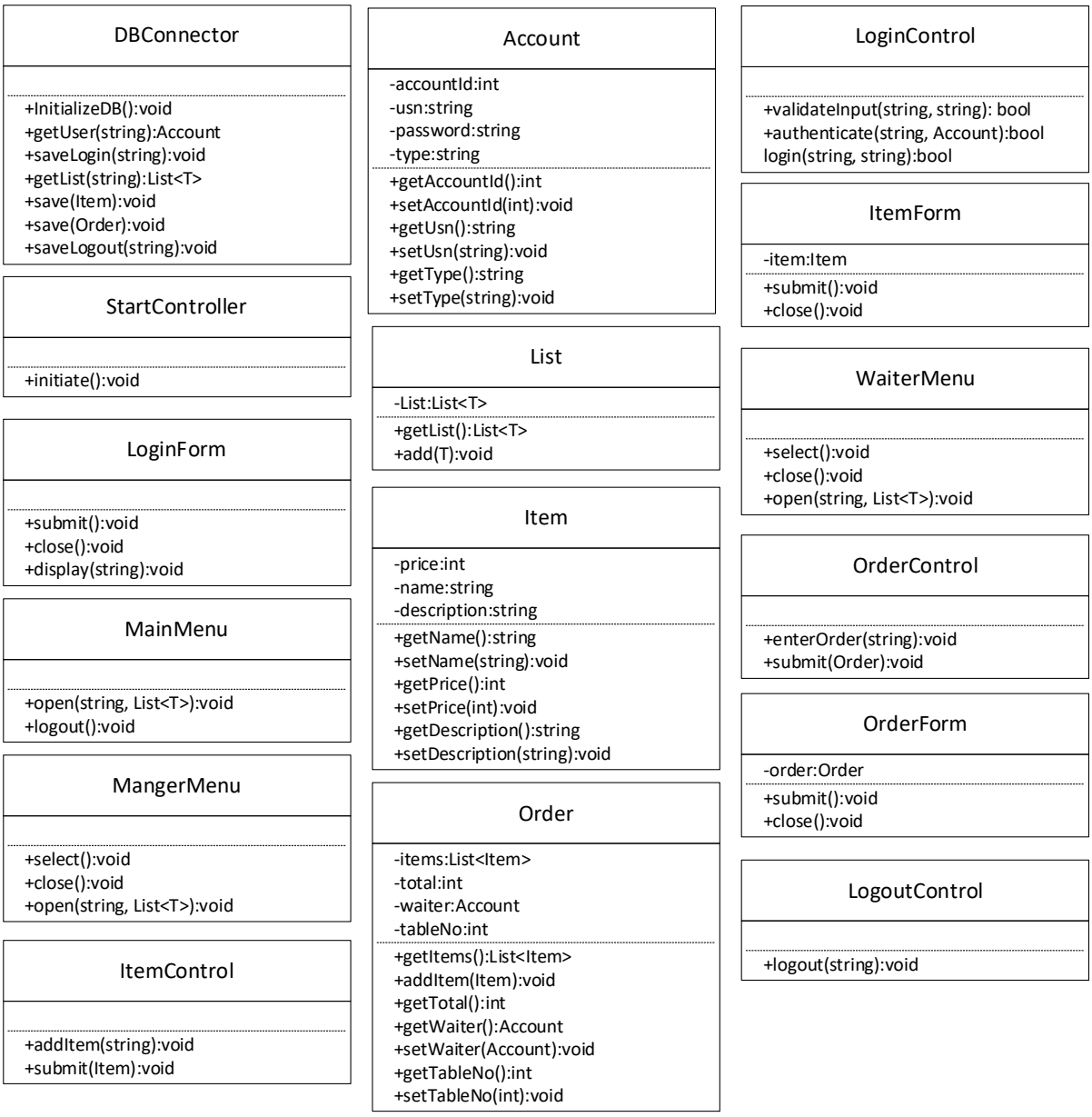


Figure 4.5 Class Diagram

4.3 DATABASE DESIGN

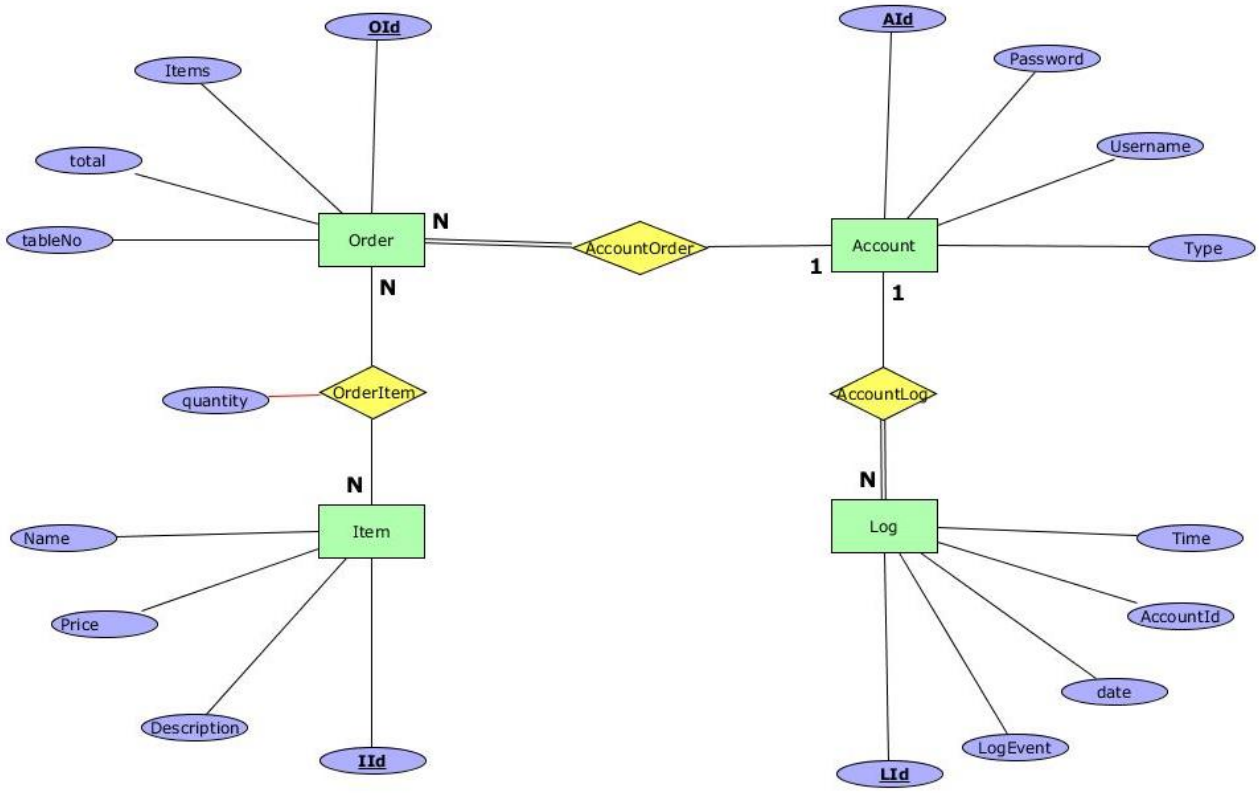


Figure 4.6: ER Diagram

Relational Model

ORDER (OId , total, items, tableNo, AId[FK])
ACCOUNT(AId [PK], Password, Username, Type)
LOG(LId , LogEvent, date, time, AId[FK])
ITEM(IId, Description, Name, Price)
ORDERITEM(id, IId [FK] , quantity, OId [FK])

5 SYSTEM DESIGN

5.1 SUBSYSTEM DECOMPOSITION

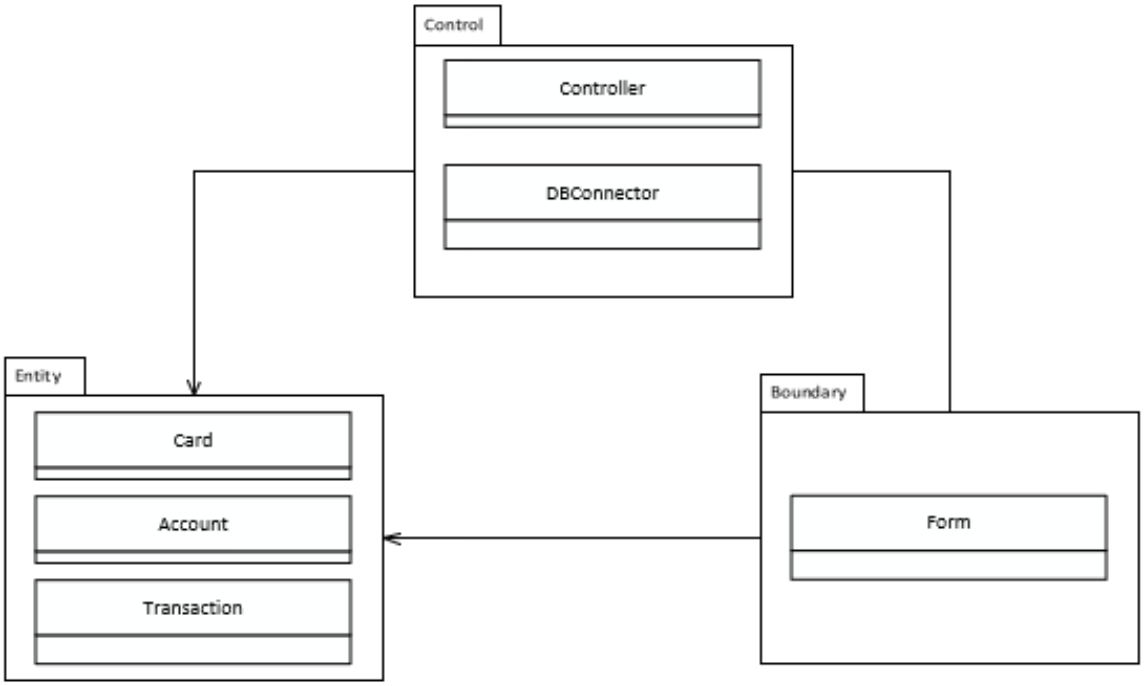


Figure 5.1: Subsystems

6 APPENDIX

6.1 APPENDIX A – SOURCE CODE

```
public class Account
{
    private int accountId;
    private string username;
    private string password;
    private string type;

    public Account(int accountId, string username, string password, string
type)
    {
        setAccountId(accountId);
        setUsername(username);
        setPassword(password);
        setType(type);
    }

    public int getAccountId() {
        return accountId;
    }

    public void setAccountId(int accountId) {
        accountId > 0
            ? this.accountId = accountId
            : throw new ArgumentException("Invalid account id");
    }

    public string getUsername() {
        return username;
    }

    public void setUsername(string username) {
        this.username = username;
    }

    public string getPassword()
    {
        return password;
    }

    public void setPassword(string password)
    {
        this.password = password;
    }

    public string getType() {
        return type;
    }

    public void setType(string type) {
        if(type.ToLowerCase().equals("waiter"))
        {
            this.type = "WAITER";
        }
        else if (type.ToLowerCase().equals("manager"))
        {
            this.type = "MANAGER";
```

```
        }
        else
        {
            throw new ArgumentException("Invalid account type");
        }
    }
}
```

```
public class Item
{
    private double price;
    private string name;
    private string description;

    public Item (double price, string name, string description)
    {
        setPrice(price);
        setName(name);
        setDescription(description);
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        price > 0
            ? this.price = price
            : throw new ArgumentException("Invalid item price");
    }

    public string getName()
    {
        return name;
    }

    public void setName(string name)
    {
        this.name = name;
    }

    public string getDescription()
    {
        return description;
    }

    public void setDescription(string description)
    {
        this.description = description;
    }

}
```

```
public class Order {
    private List<Item> items;
    private double total;
    private Account waiter;
    private int tableNo;

    public Order(List<Item> items, Account waiter, int tableNo)
    {
        this.items = items;
        total = calculateTotal();
        setWaiter(waiter);
        setTableNo(tableNo);
    }

    public Order(Account waiter, int tableNo)
    {
        items = new List<Item>();
        setWaiter(waiter);
        setTableNo(tableNo);
    }

    private int calculateTotal()
    {
        foreach(Item item in items)
        {
            total += item.getPrice();
        }
    }

    public List<Item> getItems()
    {
        return items;
    }

    public void addItem(Item item)
    {
        items.Add(item);
        total = calculateTotal();
    }

    public double getTotal()
    {
        return total;
    }

    public Account getWaiter()
    {
        return waiter;
    }

    public void setWaiter(Account waiter)
    {
        this.waiter = waiter;
    }

    public int getTableNo()
    {
        return tableNo;
    }

    public void setTableNo(int tableNo)
```

```
    {
        tableNo > 0
            ? this.tableNo = tableNo
            : throw new ArgumentException("Invalid table number");
    }

}
```